Reighfer Joshua V. Bondoc

BSIT-4A

PDC 50

ShopSmart Live Catalog Search Through the API

Repository link :https://github.com/reighbabs27/PDC50_ShopSmart.git

Architectural Path & Implementation Workflow When a shopper types into the search box on ProductsPage.jsx, the useEffect hook triggers a call to api.getProducts(search.trim()) in api.js, sending an HTTP GET request to products.js, Prisma translates this into a PostgreSQL query filtering for active items (matches either the product name or description case-insensitively using mode: 'insensitive' . To prevent race conditions during rapid typing or sudden clearing, the React effect maintains an active cancellation flag in its cleanup function, ensuring that any slow or outdated network response is discarded and only the most recent request updates the /api/products?search=... . On the Express server in active: true) where the search term products state. The category choices stay available even when search returns zero or one result because categories are accumulated in a dedicated React state (categories) rather than being derived strictly from the current filtered search results. Finally, client-side category filtering and price or name sorting are smoothly applied to the current server results so shoppers can continue refining their view without extra round-trips.

Challenge 1: Server Search (Express & Prisma)

The server route at backend/src/routes/products.js was upgraded to query both condition, preserving active product filtering (name and description via Prisma's active: true) and ascending order by id.

```
// backend/src/routes/products.js
router.get('/', async (request, response, next) => {
  try {
    const search = String(request.query.search || '').trim();
    const products = await prisma.product.findMany({
      where: {
        active: true,
        ...(search ? {
          OR: [
            { name: { contains: search, mode: 'insensitive' } },
            { description: { contains: search, mode: 'insensitive' } }
          ]
        } : {})
      },
      orderBy: { id: 'asc' }
    });
    response.json({ products: products.map((p) => ({ ...p, price: Number(p.price) })) });
  } catch (error) { next(error); }
});
```



Challenge 2 & 3: Live Frontend Search & Stable Catalog Controls

In frontend/src/pages/ProductsPage.jsx , the client connects directly to the backend search API while preserving stable category options and sorting: Race Condition Prevention: Handled via let active = true and cleanup function return () => { active = false; }; in useEffect([search]) . Category Dropdown Stability: Categories are maintained in a stateful set useState(['All']) that accumulates seen categories, preventing options from disappearing when search returns few or zero products. Instant Controls: Category filtering and sorting apply in-memory on the current server results.

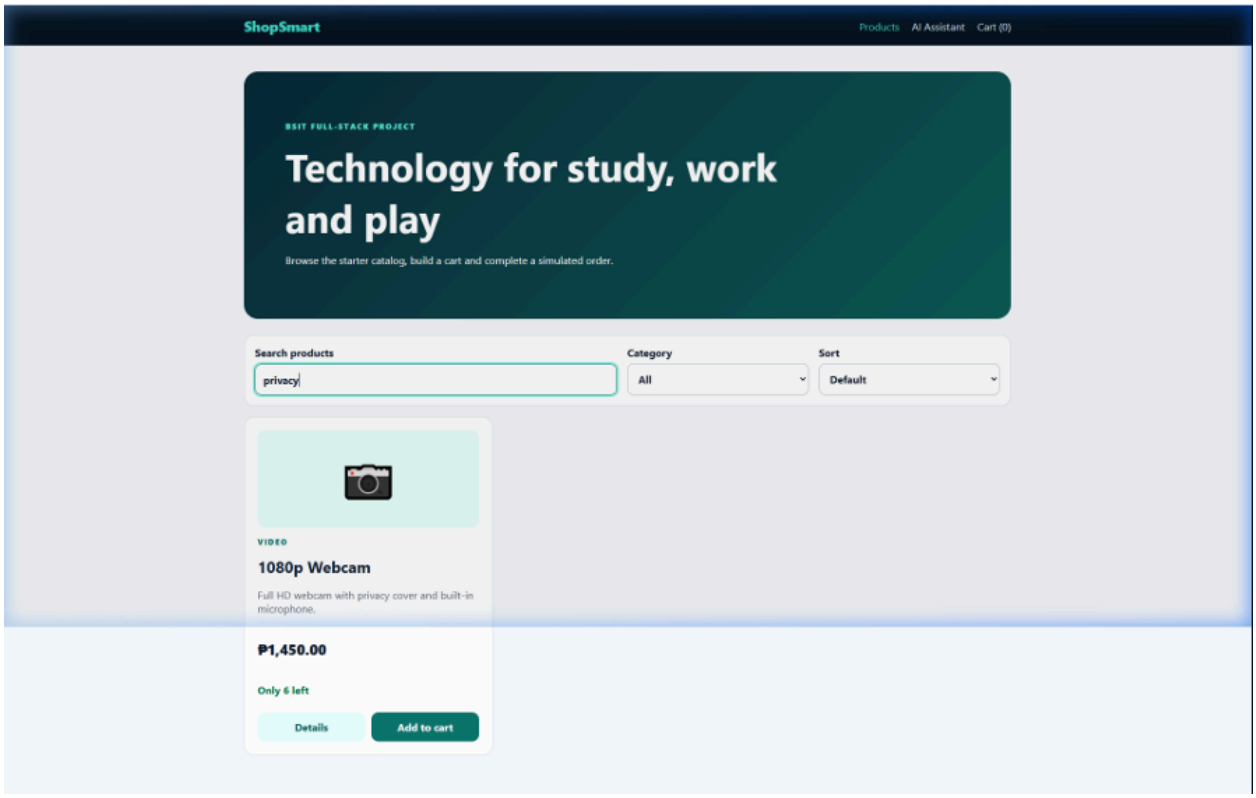
```
// frontend/src/pages/ProductsPage.jsx
export default function ProductsPage() {
  const [products, setProducts] = useState([]);
  const [categories, setCategories] = useState(['All']);
  const [search, setSearch] = useState('');
  const [category, setCategory] = useState('All');
  const [sort, setSort] = useState('default');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

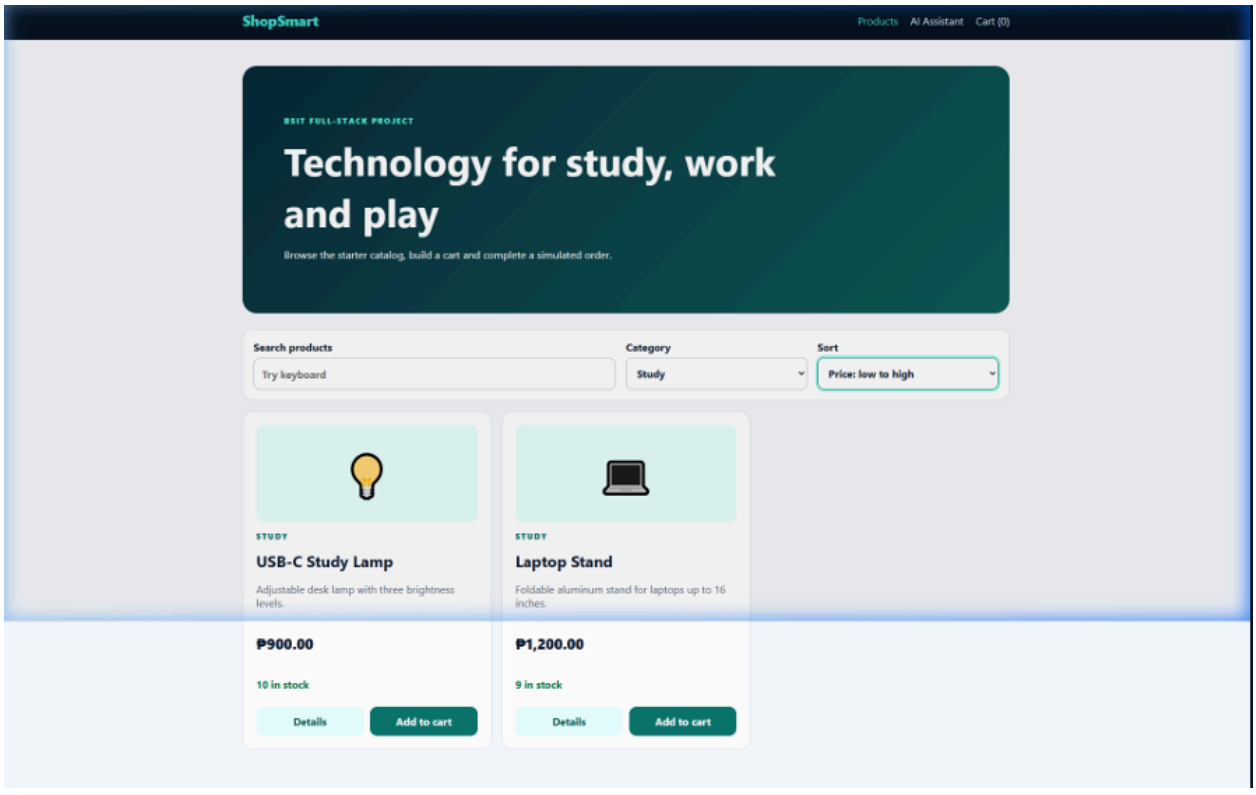
  useEffect(() => {
    let active = true;
    setLoading(true);
    setError('');

    api.getProducts(search.trim())
      .then((data) => {
        if (!active) return;
        setProducts(data.products);
        setCategories((prev) => Array.from(new Set([...prev, ...data.products.map((p) => p.category)])));
      })
      .catch((caught) => {
        if (!active) return;
        setProducts([]);
        setError(caught.message);
      })
      .finally(() => {
        if (!active) return;
        setLoading(false);
      });

    return () => { active = false; };
  }, [search]);

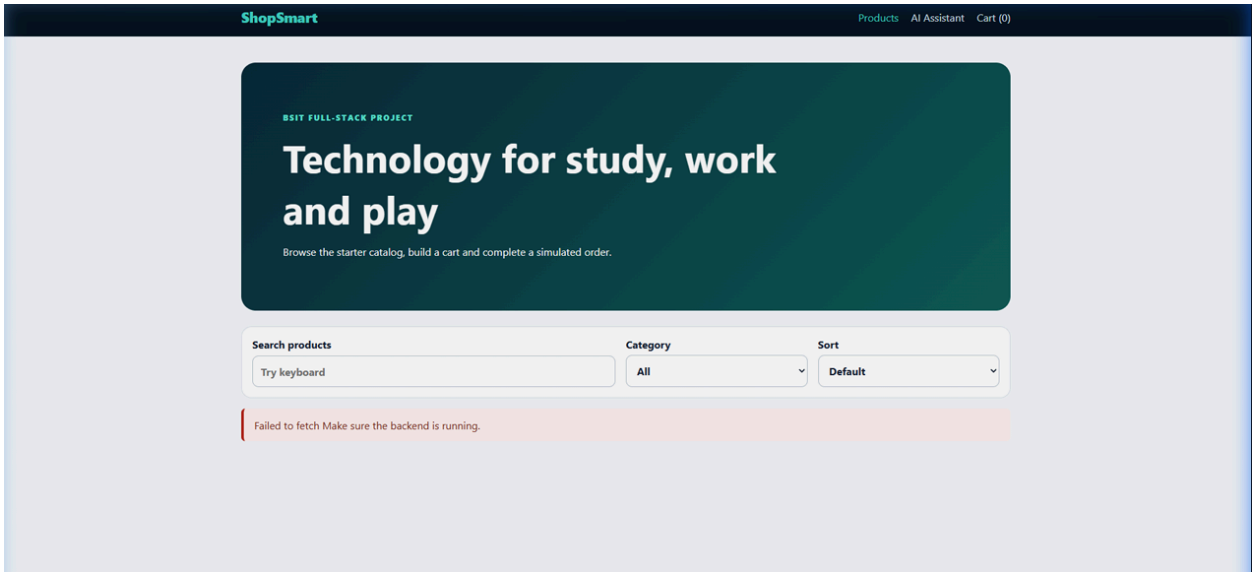
  const visibleProducts = products.filter((product) => category === 'All' || product.category === category);
  const displayedProducts = [...visibleProducts].sort((a, b) => {
    if (sort === 'price-asc') return Number(a.price) - Number(b.price);
    if (sort === 'price-desc') return Number(b.price) - Number(a.price);
    if (sort === 'name-asc') return a.name.localeCompare(b.name);
    return 0;
  });
  // ...
```





Server Failure Handling & Recovery Check

When the backend server is stopped, searching immediately displays a clean, user-friendly error message. Upon restarting the backend, the catalog recovers cleanly with full functionality.



Complete Full-Stack Verification (npm run check)
The full verification pipeline runs ESLint on both frontend and backend workspaces, unit tests via Vitest and Node Test Runner, and Vite production bundle compilation. All 4 phases passed with zero errors:

```
> shopsmart-starter@1.0.0 check
> npm run lint --workspace frontend && npm run lint --workspace backend && npm run test && npm run build

> shopsmart-frontend@1.0.0 lint
> eslint .

> shopsmart-backend@1.0.0 lint
> eslint .

> shopsmart-starter@1.0.0 test
> npm run test --workspace frontend && npm run test --workspace backend

> shopsmart-frontend@1.0.0 test
> vitest run
 RUN  v3.2.7 C:/Users/Reighh/Downloads/ShopSmart-Starter-main/frontend
 ✓ src/utils/cart.test.js (5 tests) 3ms
 Test Files  1 passed (1) | Tests 5 passed (5)

> shopsmart-backend@1.0.0 test
> node --test
✓ parseBudget extracts a peso budget (0.7353ms)
✓ rankProducts respects the budget (0.6671ms)
✓ assistant response contains traceable sources (0.2021ms)
✓ a valid classroom order passes validation (2.3692ms)
✓ duplicate products fail order validation (0.2902ms)
✓ an oversized assistant question fails validation (0.334ms)
ℹ tests 6 | pass 6 | fail 0 | duration_ms 146.7484

> shopsmart-starter@1.0.0 build
> npm run build --workspace frontend
vite v7.3.6 building client environment for production...
✓ 52 modules transformed.
dist/index.html                   0.49 kB | gzip:  0.30 kB
dist/assets/index-C_E__iQw.css    4.45 kB | gzip:  1.52 kB
dist/assets/index-MpUDZ8SY.js   273.38 kB | gzip: 86.52 kB
✓ built in 975ms
```